

TRUE

```
$ echo AAABBBB | sed 's/AAABBBB/ s/\(.*\)B\1/; t again ; s/^$/TRUE/; t ; s/./FALSE/'
```

FALSE

Let's go through the above *sed* script, part by part.

<code>s/A\(.*)B\1/</code>	Removes the outer A and B. On the first operation, AAABBBB will become AABBB.
<code>:again</code>	A label that is used as the target of the <i>t</i> jump operator.
<code>t again</code>	If the substitution operation <code>s/A\(.*)B\1/</code> succeeds, it jumps to the label <i>again</i> , and executes from there.
<code>s/^\$/TRUE/; t ;</code>	If the current pattern space is empty, set the pattern space to <i>TRUE</i> . The <i>t</i> ; says that if the substitution operation succeeds, jump to the end of the script.
<code>s/./FALSE/</code>	If the previous " <i>t</i> " test fails, replace the pattern space with <i>FALSE</i> .

A few cool scripts

We have gone through the essential components and features of *sed*. Now let us go through a few nice examples of *sed* scripts.

Emulating the *tac* command

The *tac* command is used to print input lines in the reverse order. You can perform the reversal of lines by pushing the lines in a stack data-structure, and when it reaches the end, start popping and printing lines, one by one. Using *sed*, we can manipulate stack operations with *h* and *G*.

```
sed -n '1G; h ; ${ x; p }' filename
```

<code>1G</code>	Execute <i>G</i> when it is not the first line.
<code>h</code>	Copy the contents of the pattern buffer to the hold buffer.
<code>\${ x; p }</code>	When the last line is reached, exchange the hold buffer contents to the pattern buffer, and print them out.

When *G* is applied, it appends the hold buffer contents to the pattern space. It is again pushed to the hold buffer, and the next time, again appended with the pattern space. Thus, a stack push operation is implemented.

Checking for palindrome

```
$ echo malayalam | sed 's/AAABBBB/ s/\(.\)\(.*\)1\2/; t again ; s/^\?$/TRUE/; t ; s/./FALSE/'
```

TRUE

This palindrome-check script is a slight modification of the example given in the flow controls with the labels section. Try to dissect it yourself.

Reversing words in a line

```
$ echo this is a line | sed '/\n/!G; s/\(w*\) \(.*\n\)/&\2\n/; //D; s/\n//;'
```

line a is this

In this script, we again use a type of stack push-and-pop operation. Let us dissect the script. Initially, if the contents of the pattern space do not contain *\n*, it appends the contents of the hold buffer to the pattern space. The hold buffer initially contains the *\n* character. Then the current pattern space is replaced with the current pattern space appended with the first word, and the rest of the text in the reversed order. Subsequently *//* matches the current pattern space if not empty, and deletes up to the first occurrence of *\n*, and the next iteration begins. At the end, an additional *\n* is replaced, and printed to get reversed words in the line.

Emulating the *tail* command

The *tail* command (by default) prints the last 10 lines of a text file. Let us look at how to keep 10 lines in the memory of *sed* as it processes lines:

```
$ seq 100 | sed '$q; :start; N ; 11,$D ; b start'
```

In this script, *N* is used to append newly read lines to the pattern space. It is iterated in a loop using a branch with the label *start*. From lines 11 to the last line, it deletes a line at the start of the pattern space, up to the first *\n*, so that the total lines in the pattern space will always be 10. Hence, when *sed* reaches the last line, it quits (*\$q*) and prints the pattern space.

We have covered most of the basic use of *sed*, with the text processing capabilities of the stream editor. For more information, there is a *sed* one-liner collection, written by the community at <http://sed.sourceforge.net/sed1line.txt>. Happy hacking till we meet again! 

By: Sarath Lakshman

The author is a hacktivist of Free and Open Source Software from Kerala. He loves working on the GNU/Linux environment, and contributes to the Pardus Linux distro project. He has recently authored the Linux Shell Scripting Cookbook, which gives insights on shell scripting through 119 recipes. He can be reached via his website, <http://www.sarathlakshman.com>.